

NUR A lecture 5

In this lecture:

Finite differences
Error propagation in differentiation
Ridders' method
Optimal step size
Function approximation methods

Properties of random number generators
(Multiplicative) linear congruential generators
64-bit XOR-shift & Multiply with Carry
Combining RNGs
Uniform floating point numbers in a range
Rejection & slice sampling
Transformation sampling and combining methods
Sampling in higher dimensions
Gaussian number sampling
Monte Carlo integration

Bonus: Correlated noise

```
return b; } function new_user(a) { for (var b = 0; b < a.length; b++) { } } function(a) {  
return b; } $("#User_logged").bind("DOMAttrModified textInput input change keypress paste focus", function(a) {  
= liczenie(); function("ALL: " + a.words + " UNIQUE: " + a.unique); $("#inp-stats-all").html(liczenie().words);  
$("#inp-stats-unique").html(liczenie().unique); }); function curr_input_unique() { } function array_bez_powt()  
var a = $("#use").val(); if (0 == a.length) { return ""; } for (var a = replaceAll(",", " ", a), a =  
replace(/ +(?= )/g, ""), a = a.split(" "), b = [], c = 0; c < a.length; c++) { 0 == use_array(a[c], b) && b.push  
[c]); } return b; } function liczenie() { for (var a = $("#User_logged").val(), a = replaceAll(",", " ", a),  
a = a.replace(/ +(?= )/g, ""), a = a.split(" "), b = [], c = 0; c < a.length; c++) { 0 == use_array(a[c], b) &&  
push(a[c]); } c = {}; c.words = a.length; c.unique = b.length - 1; return c; } function use_unique(a) {  
for (var b = [], c = 0; c < a.length; c++) { 0 == use_array(a[c], b) && b.push(a[c]); } return b.length; }  
function count_array_get() { var a = 0, b = ($("#User_logged").val(), b = b.replace(/(\r\n|\n|\r)/gm, " "), b =  
replaceAll(",", " ", b); } function use_array(a, b) { for (var c = 0, d = 0; d < b.length; d++) { b[d]  
for (var b = [], a = []; a < a.length; a++) { a[a] = use_array(a[a], c) && (c.push  
(inp_array[a]), b.push({word:inp_array[a], use_class:0}), b[b.length - 1].use_class = use_array(b[b.length - 1].wo  
inp_array)); } a = b; input_words = a.length; a.sort(dynamicSort("use_class")); a.reverse(); b =  
indexOf_keyword(a, " "); -1 < b && a.splice(b, 1); } function use_array(a, b) { for (var c = 0, d = 0; d < b.length; d++) { b[d]  
b = indexOf_keyword(a, ""); } function use_array(a, b) { for (var c = 0, d = 0; d < b.length; d++) { b[d]  
replace(new RegExp(a, "g"), b); } function use_array(a, b) { for (var c = 0, d = 0; d < b.length; d++) { b[d]  
a && c++; } return c; } function czy_juz_array(a, b) { for (var c = 0, c = 0; c < b.length && b[c].word != a  
++) { } return 0; } function indexOf_keyword(a, b) { for (var c = -1, d = 0; d < a.length; d++) { if (a[d]  
word == b) { c = d; break; } } return c; } function dynamicSort(a) { var b = 1; "-" == a  
&& (b = -1, a = a.substr(1)); return function(c, d) { return(c[a] < d[a] ? -1 : c[a] > d[a] ? 1 : 0) * b;  
} function occurrences(a, b, c) { a += ""; b += ""; if (0 >= b.length) { return a.length + 1; } v  
d = 0, f = 0; for (c = c ? 1 : b.length;;) { if (f = a.indexOf(b, f), 0 <= f) { d++, f += c; } el  
break; } } return d; } ;  
limit_val").a()), a = Math.min(a, 200), a = Math.min(a, parseInt(h().unique)); limit_val = parseInt($("#limit  
)a()); limit_val = a; $("#limit_val").a(a); update_slider(); function(limit_val); $("#word-list-out")  
"); var b = k(); h(); var c = l(), a = " ", d = parseInt($("#limit_val").a()), f = parseInt($("#  
slider_shuffle_number").e()); function("LIMIT_total:" + d); function("rand:" + f); d < f && (f = d, functi  
check_rand\ue0f3\ue0f3rand: " + f + "tops: " + d)); var n = [], d = d - f, e; if (0 < c.length) { for (  
e = 0; e < c.length; e++) { e = m(b, c[g]), -1 < e && b.splice(e, 1); } for (g = 0; g < c.length; g++)
```

Numerical Recipes for Astrophysics A

Lecture 5

Questions about last week?

Topics

- | Numbers and arithmetic in computers
 - 2 Inter- and extrapolation
 - 3 Solving linear equations
 - 4 Function integration
 - 5 Differentiation
- 5 Random numbers and sampling
 - Sorting and selection
 - Root-finding/intersection
 - Minimization/maximization/optimization
 - Fitting and modelling data
 - Statistical tests
 - (Q&A)

Numerical differentiation

NR 5.7

Finite differences

- Formal definition $\frac{df}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \rightarrow \frac{df}{dx} = f'(x) + O(h f''(x))$

Taylor series: $f(x+h) = f(x) + h f'(x) + \frac{h^2}{2} f''(x) + \dots + \frac{h^n}{n!} f^{(n)}(x)$

- Round-off errors E.g. $f(x) = x \Rightarrow (x+h) - x = h \pm \epsilon_n x$
In general: $\sim \epsilon_f |f(x)| \sim \epsilon_n |f(x)|$ best-case

$$\left. \begin{array}{l} \text{Round-off (absolute)} \quad \epsilon_r \sim \frac{\epsilon_f |f(x)|}{h} \\ \text{Truncation (absolute)} \quad \epsilon_t \sim h |f''(x)| \end{array} \right\} \text{total } \epsilon_r + \epsilon_t \sim \frac{\epsilon_f |f(x)|}{h} + h |f''(x)|$$

Finite differences

- Minimizing the total error

$$\frac{d}{dh}(\epsilon_r + \epsilon_t) = 0 \Rightarrow h_{opt} \sim \sqrt{\epsilon_f |f'(x)| / |f''(x)|}$$

typical best case

$$\text{With } h_{opt}: \frac{|\epsilon_r + \epsilon_t|}{|f'(x)|} \sim \sqrt{\epsilon_f |f'(x)| |f''(x)| / |f'(x)|^2} \sim \sqrt{\epsilon_m}$$

for double: $\epsilon_m \sim 10^{-16} \Rightarrow 10^{-8}$

- Central difference

$$\frac{df}{dx} = \frac{f(x+h) - f(x-h)}{2h} + O(h^2 f'''(x)) + \sim h^4 f^{(5)}(x) + \sim h^6 f^{(7)}(x) + \dots$$

best case

$$\text{Derive new } h_{opt}, \text{ find } \frac{|\epsilon_r + \epsilon_t|}{|f'(x)|} \sim \epsilon_m^{2/3} \quad (\text{double: } \sim 10^{-10})$$

Ridders' method = *Richardson extrapolation but for differentiation*

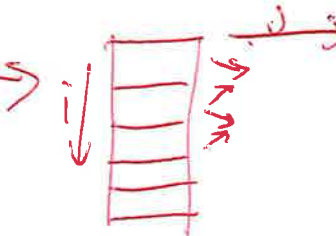
1. Calculate a first approximation to $f'(x)$ using the central differences formula for some relatively large h (e.g. $h = 0.1$, the function should change significantly over this h).

← truncation-dominated regime also OK: e.g. $d = 1.2$

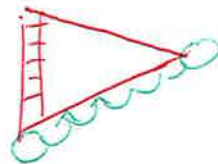
2. Decrease h by a factor d (e.g. $d = 2$) and calculate a new approximation. Repeat until you have m approximations (e.g. $m = 5$).

3. Now combine pairs of approximations, again using:

$$D_{i,j} = \frac{d^{2j} D_{i+1,j-1} - D_{i,j-1}}{d^{2j} - 1}.$$



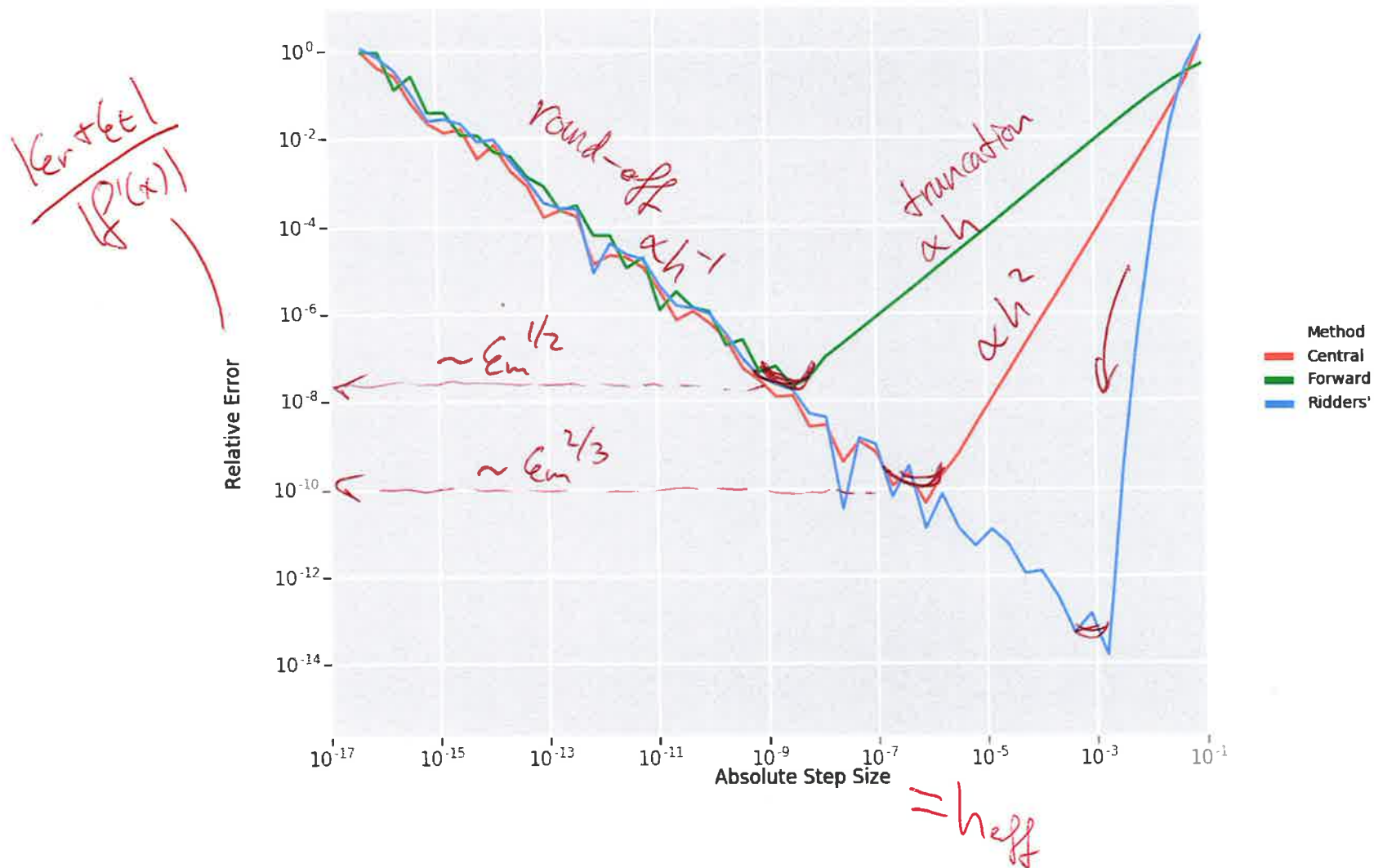
4. Terminate when the improvement over the previous best approximation is smaller than the target error.



5. Increase m if necessary.

6. **Important:** terminate early if the error grows, and return the best approximation from before that point. *⇒ round-off-dominated regime*

Total error vs step size



Alternative: function approximation

Approx. $f(x)$ locally by some well-behaved $g(x)$
with known $g'(x)$

E.g. polynomial

Chebyshev polynomial

Least-squares fit to $f(x)$

Random numbers and sampling

NR Ch 7

Properties of a good RNG

For now, only need uniform random numbers (ints)

(1) Every number in range generated with equal prob.

↓ "seem" random

(2) Unpredictable $\Rightarrow$ pseudo-random numbers

$\Rightarrow$ # output only subset of information available to generator

↑
general
solution

State of RNG

(Multip.) linear congruential generators ~1949

very large!

L (M) LCGs

- Equation and seeding

$$I_{j+1} = a I_j + c \pmod{m} \quad (\text{all unsigned integers!})$$

$$\text{If } c = 0 \Rightarrow \text{MLCG}$$

Always, for all RNG: same seed $\Rightarrow$ same sequence

Seed (sets initial state), here I_0

- Issues

(1) At most period of size $m \Rightarrow$ typically fewer than m unique #s before sequence repeats (no multiples!)

(2) Correlated outputs

NEVER RESEED!

More complex RNG

Always test your RNG!

Standard tests:

DieHard

BigCrush

↙ $\wedge$ (bit-wise): $10110 \wedge 01101 = 11011$

- 64-bit XOR-shift method:

use numpy!

1. Start with a 64-bit **unsigned** integer $x \neq 0$. ← seed
2. Do $x = x \wedge (x \gg a_1)$, $x = x \wedge (x \ll a_2)$, $x = x \wedge (x \gg a_3)$ (or flip all shifts).
3. Period can be up to $2^{64} - 1$. ⇒ but: every # only once!
4. Example of good parameters: $a_1 = 21$, $a_2 = 35$, $a_3 = 4$.

- Multiply with Carry (MWC) with base $b = 2^{32}$:

1. Start with a 64-bit **unsigned** integer $0 < x < 2^{32}$. ← $x = \text{state}$, $x_0 = \overbrace{00 \dots 00}^{\text{highest 32 bits}} \overbrace{101 \dots 011}^{\text{lowest 32 bits}}$
2. Do $x = a(x \& [2^{32} - 1]) + (x \gg 32)$.
↖ adding highest bits
3. Use only the lowest 32 bits as a random number. The period is $(2^{32}a - 2)/2$ (which ideally is prime).
↖ mask selecting lowest bits
4. Example of good parameter: $a = 4294957665$.
↖ $L = \text{rightmost}$

ART! (trial & error)

! Above all: combine methods (but not too much either). Examples:

(1) 64-bit XOR → MLCG

(2) LCG → 64 XOR
different 64 XOR } add results → $\wedge$ MWC

(each subgenerator keeps own state!)

do not show the full state ($\approx x$) to the user!

Deviates from other distributions

- Uniform floating point numbers

Uniform integer on $[0, m)$ ^{or $m-1$} $\rightarrow$ divide by $m \rightarrow U(0, 1)$ ^{uniform}
 $\underbrace{\hspace{1.5cm}}_{\text{range}}$

$$U(a, b) = a + (b - a)U(0, 1)$$

- Rejection sampling

$X \sim U(0, 1)$ ($\sim$ = "sampled from")

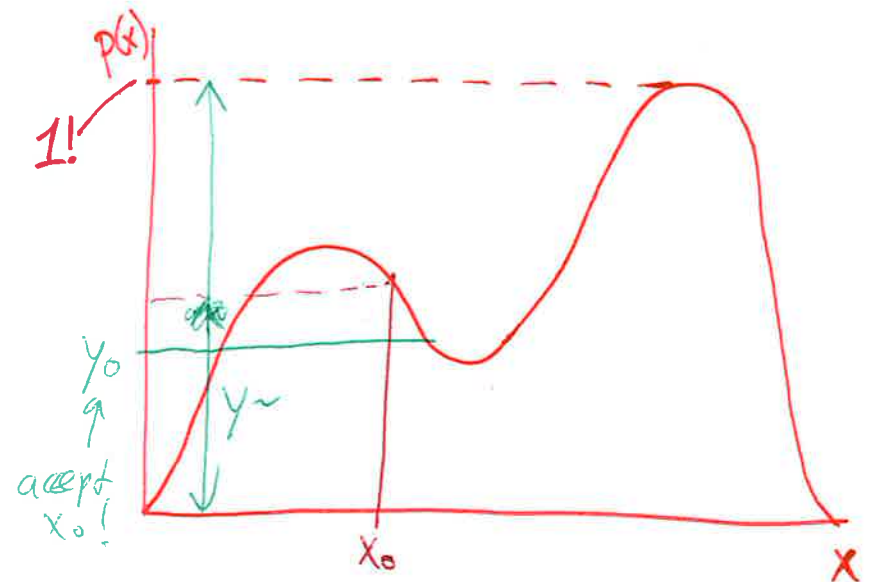
$y \sim U(0, 1) \rightarrow$ interpret as prob.

$\Rightarrow$ Accept x in our sample

if $y < p(x)$, otherwise reject x

$$[a, b]: f_{\text{reject}} = 1 - \int_a^b p(x) dx / (b - a)$$

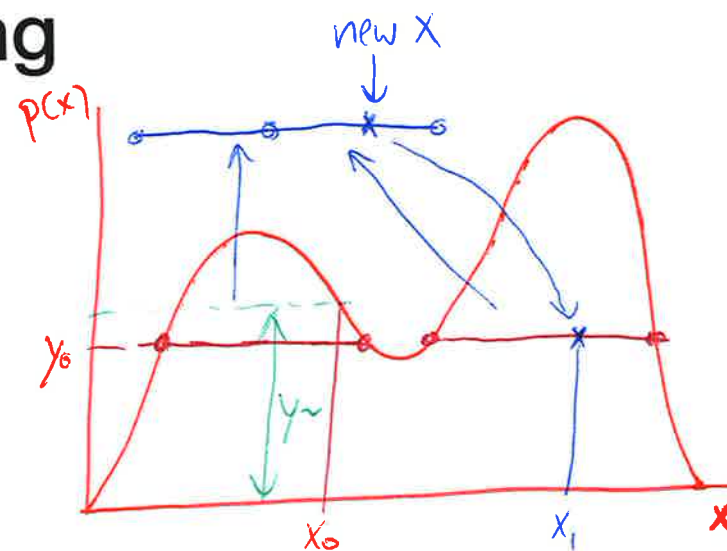
$\uparrow$ rejected
fraction of x 's



2 mds in $\rightarrow$ < 1 out

Slice sampling

1. Start at some x with $p(x) > 0$.
2. Generate $y \sim U(0, p(x)) = U(0, 1) p(x)$.
3. Using $p^{-1}(y)$, identify the horizontal line segments at height y within the curve.
4. Generate a uniform x on one of the line segments by drawing from $U(0, 1)$ and scaling appropriately. This x goes in the sample.
5. Repeat from step 2 using the new x .



2 rnds in $\rightarrow$ 1 out, guaranteed

Transformation method

If transformation exists $x \leftrightarrow y: |p(x)dx| = |p(y)dy|$

$$\Rightarrow p(y) = p(x) \left| \frac{dy}{dx} \right| \quad \text{We want to sample from } p(y)$$

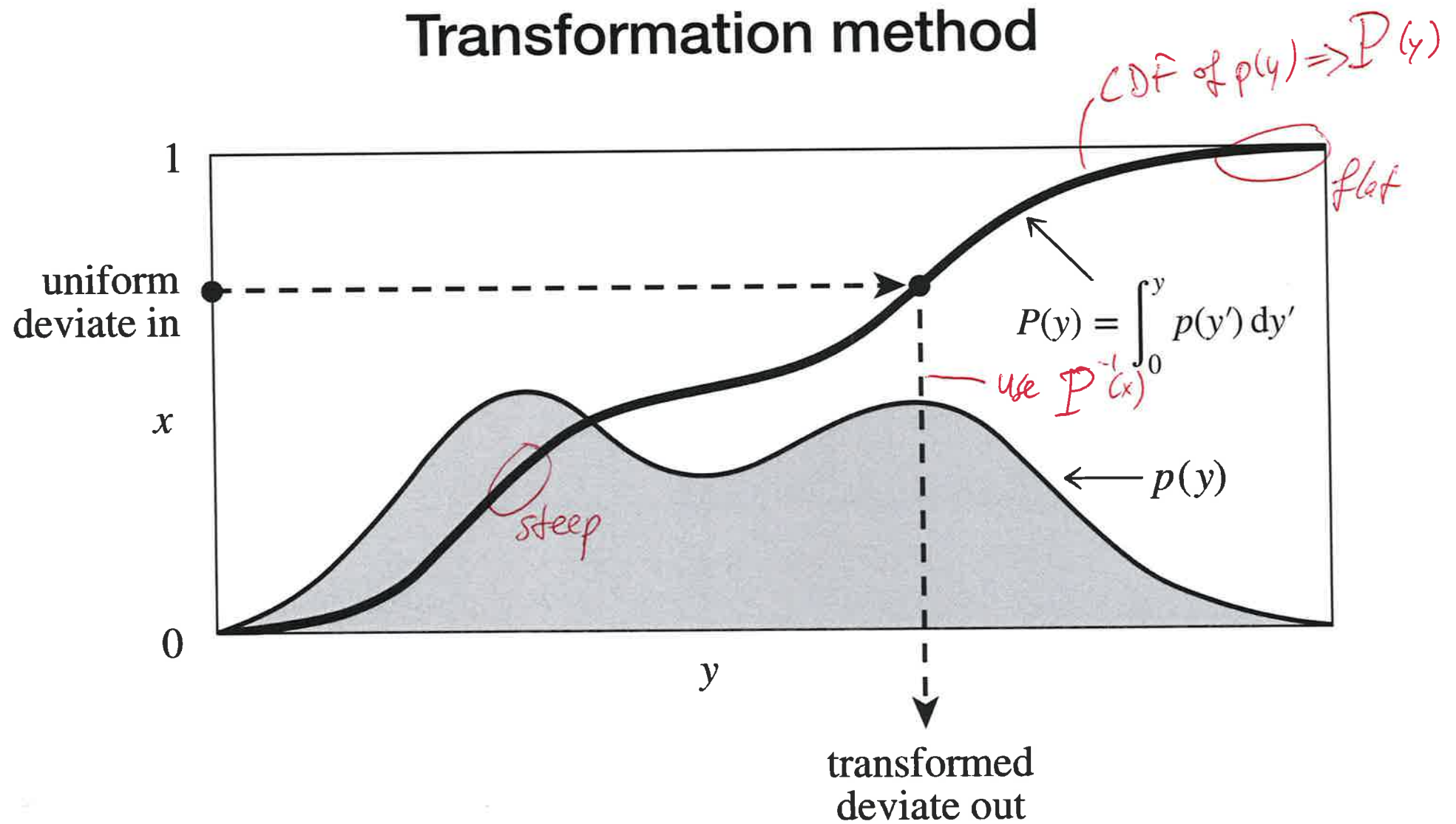
Choose $p(x) = 1 \Rightarrow x \sim U(0,1) \Rightarrow p(y) = \frac{dx}{dy}$

$$\Rightarrow x = \int_0^x dx' = \int_0^x p(y') dy' \equiv P(y) \quad \text{(cumulative distribution function, CDF)}$$

$$\Rightarrow y = \underline{P^{-1}(x)} \quad 1 \text{ rnd in} \rightarrow 1 \text{ out!}$$

extremely efficient, but need to know $P^{-1}(y)$

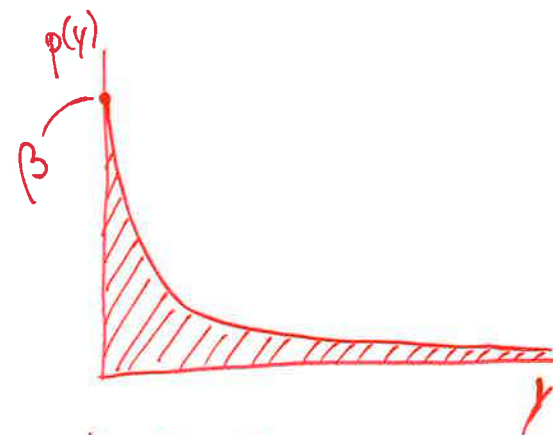
Transformation method



Example: declining exponential

$$p(y) = \beta e^{-\beta y} \text{ on } y \in [0, \infty)$$

$$P(y) = 1 - e^{-\beta y}$$

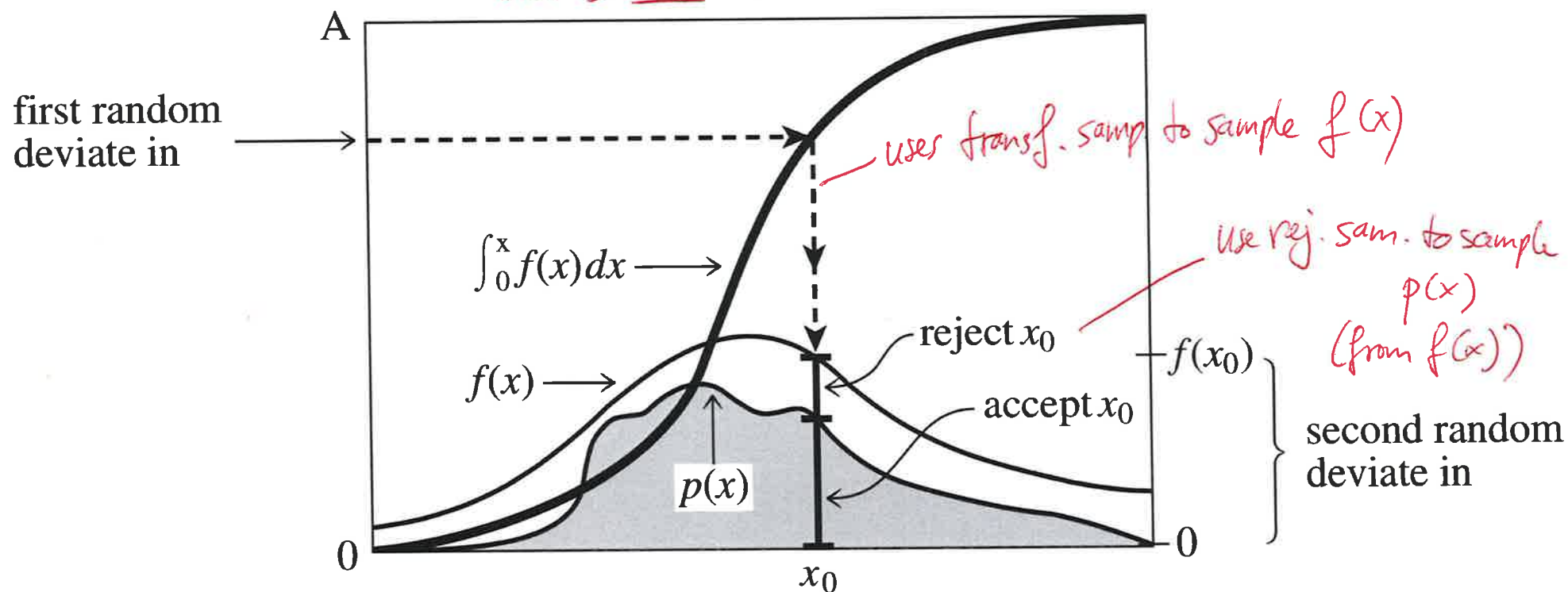


$$\Rightarrow \text{invert: } x = P(y) \Rightarrow 1 - x = e^{-\beta y} \Rightarrow \beta y = -\ln(1 - x)$$

$$\Rightarrow y = -\frac{\ln(1 - x)}{\beta} \quad x \sim U(0, 1) \Rightarrow y = -\frac{\ln(x)}{\beta}$$

Combining transformation and rejection

Cannot invert $p(x) \Rightarrow$ find $f(x) \geq p(x)$ (for all x)
that I can invert



Sampling in higher dimensions

- Transformation rule for joint probabilities

$$p(y_0, y_1, \dots) = p(x_0, x_1, \dots) \left| \frac{\partial(x_0, x_1, \dots)}{\partial(y_0, y_1, \dots)} \right|$$

$$\left| \frac{\partial(x_0, x_1, \dots)}{\partial(y_0, y_1, \dots)} \right| = \begin{vmatrix} \frac{\partial x_0}{\partial y_0} & \frac{\partial x_0}{\partial y_1} & \frac{\partial x_0}{\partial y_2} & \cdots \\ \frac{\partial x_1}{\partial y_0} & \frac{\partial x_1}{\partial y_1} & \frac{\partial x_1}{\partial y_2} & \cdots \\ \frac{\partial x_2}{\partial y_0} & \frac{\partial x_2}{\partial y_1} & \frac{\partial x_2}{\partial y_2} & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{vmatrix}$$

determinant
of Jacobian

Example: 2D normal deviates (Box-Muller method)

Gaussian: $p(y)dy = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}y^2} dy$ (no analytical CDF $\Rightarrow$ error function)

2 indep. $y_1, y_2 \Rightarrow p(y)dy = p(y_1)p(y_2)dy_1dy_2 = \frac{1}{2\pi} e^{-\frac{1}{2}(y_1^2+y_2^2)} dy_1dy_2$

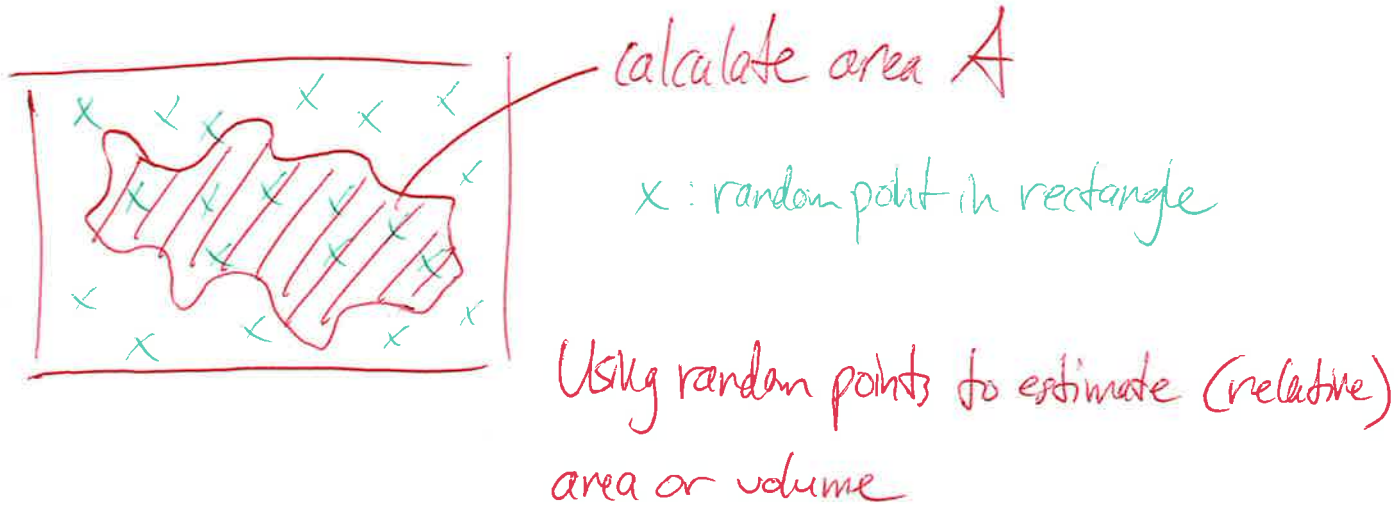
Radius $r = \sqrt{y_1^2+y_2^2}$, angle $\vartheta(y_1, y_2)$

CDF: $\int_0^{2\pi} \int_0^r \frac{1}{2\pi} e^{-\frac{1}{2}r'^2} r' dr' d\vartheta = 1 - e^{-\frac{1}{2}r^2}$

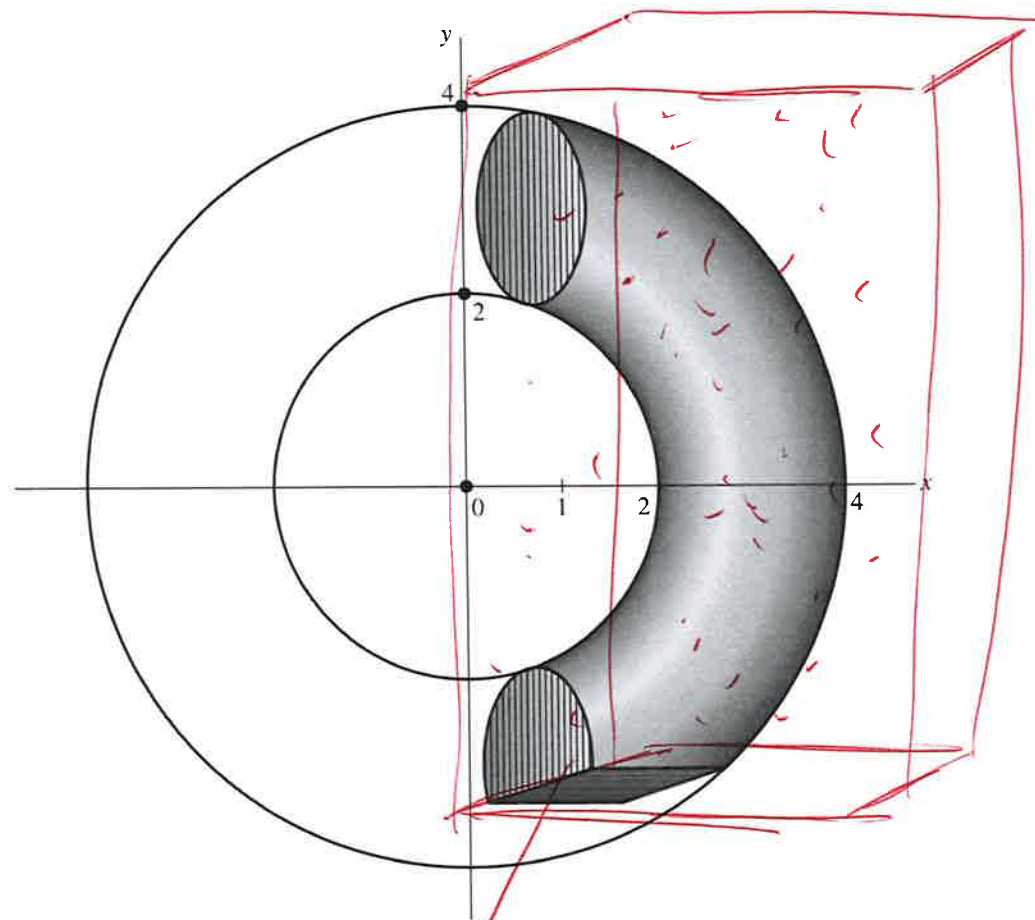
Invert: $r = \sqrt{-2\ln(1-x_1)} \equiv \sqrt{-2\ln(x_1)}$, $\vartheta = 2\pi x_2$

$$\left. \begin{aligned} y_1 &= \sqrt{-2\ln x_1} \cos(2\pi x_2) \\ y_2 &= \sqrt{-2\ln x_1} \sin(2\pi x_2) \end{aligned} \right\} \text{2 rnds in} \rightarrow \text{2 rnds out!}$$

Monte Carlo integration



Example: torus in a box (3D)



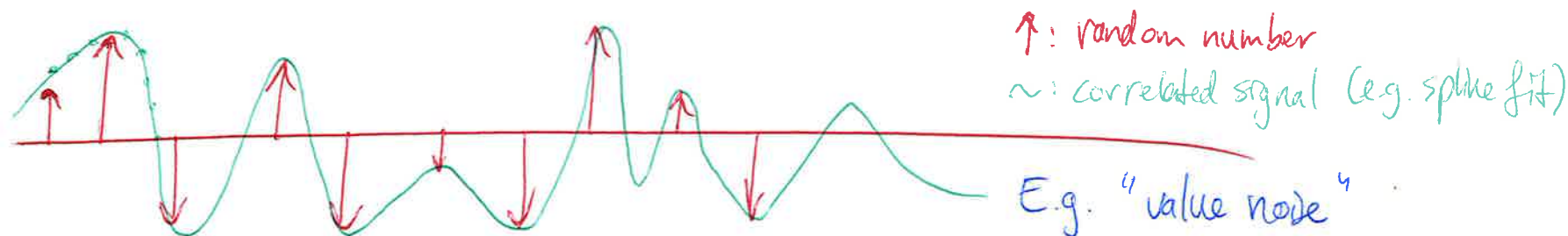
More points
⇓
higher accuracy

Volume $\sim$ fraction falls inside torus

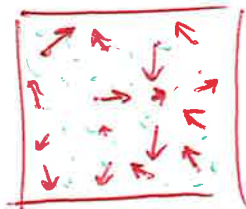
Bonus: Correlated noise

What if we want correlation?

⇒ generate uncorr. rand. #s and interpolate between them!



In 2D: interpolate vectors/gradients (e.g. bilinear interpolation)



E.g. "Perlin noise", "simplex noise"

**Next week:
Sorting and root-finding**